Requirement Analysis

“Family photo wall: Family vault”

PRESENTED BY

ARJUN MANKUSKAR(202401110038)

ATULYA MISHRA(202401110051)

VISHAL SHENDE(202401110034

Project Title: Virtual Family Photo Wall

1. Introduction

The Virtual Family Photo Wall (VFPW) aims to provide a private, creative, and centralized digital space for families to store, view, and celebrate their memories. It provides a "Family Head" (Admin) an interface to create the family space and allows "Family Members" to upload photos, add context, and view automatically generated themes.

2. Purpose

The purpose of this document is to describe the functional and non-functional requirements of the Virtual Family Photo Wall (VFPW) application. It serves as a guide for developers, testers, and stakeholders.

3. Scope

- **User Management:** The Family Head (Admin) creates the account and invites members. Members can join via secure authentication.
- **Media Management:** Users can upload photos and attach text or voice notes.
- **AI Features:** The system utilizes Generative AI to suggest creative themes for special occasions.
- **Viewing:** A responsive "Photo Wall" displays memories in a creative layout.

4. Definitions, Acronyms, and Abbreviations

- **VFPW:** Virtual Family Photo Wall[9].
- **Admin:** The user who manages the family group.
- **Firestore:** The NoSQL database used for storing data.
- **Genkit:** An AI SDK used for generating theme suggestions.

5. Overall Description

5.1 Users and Characteristics

User	Description
Admin	Creates family group, invites members, manages special dates.
Member	Joins group, uploads photos/notes, interacts with the wall.

5.2 Assumptions and Dependencies

- The system is a web application optimized for modern browsers.
- Reliance on **Firebase** for backend services (Auth, DB) and **Cloudinary** for media optimization.

- Stable internet connection required for AI generation and media streaming.

6. Functional Requirements

6.1 User and Family Account Management

- Users can register/login using **Firebase Authentication** (supporting Email/Password and Google Sign-In).
- Admins can invite family members via email link.

6.2 Photo Wall Management

- **Uploads:** Users can upload photos. High-performance uploads and optimization are handled via **Cloudinary**.
- **Metadata:** Users can add captions and voice notes. Metadata is stored in **Firestore**.
- **Management:** Users can delete or edit their own content[13].

6.3 AI & Special Days (Updated)

- **Special Dates:** Admin defines dates (e.g., Birthdays).
- **AI Theme Generation:** On special days, the system uses **Genkit (with Google Gemini)** to analyze context and suggest/generate a "Themed Wall" layout or creative title (e.g., suggesting "Golden Memories of 2023" based on photo metadata).

6.4 Notifications

- System sends notifications to members for special days or new uploads.

7. Non-functional Requirements

7.1 Performance

- **Load Time:** The Photo Wall must utilize **Next.js** server-side rendering and client components for fast initial loads.
- **Optimization:** Images must be delivered via **Cloudinary** to ensure automatic format selection (WebP/AVIF) and resizing based on device.

7.2 Security

- **Authentication:** All routes must be protected via **Firebase Authentication**.
- **Type Safety:** The codebase must use **TypeScript** to prevent runtime errors and ensure data integrity.

7.3 Usability

- **UI Components:** The interface must be built using **ShadCN UI** to ensure accessibility, consistency, and a clean, modern aesthetic.

- **Responsiveness:** Must work seamlessly on mobile, tablet, and desktop.

8. System Architecture (Updated)

This section outlines the modern tech stack utilized for VFPW.

- **Front-end Framework: Next.js**
    - Chosen for its hybrid architecture, allowing a mix of Server-Side Rendering (SSR) for performance and Client-Side Rendering (CSR) for interactivity.
- **Language: TypeScript**
    - Ensures type safety and reduces bugs during development.
- **Backend & Database: Firebase (BaaS)**
    - **Authentication:** Handles secure user sign-in (Email/Google).
    - **Firestore:** A real-time NoSQL database for storing user profiles, family relationships, and photo metadata.
    - **Firebase Storage:** Used for secure backup of raw files.
- **Media Engine: Cloudinary**
    - Handles direct client-side image uploads and delivery optimization.
- **AI Layer: Genkit + Google Gemini**
    - Powers the "Theme Suggestion" feature by analyzing text/dates to propose creative wall themes.
- **UI Library: ShadCN UI**
    - Provides accessible, customizable pre-built components (Cards, Dialogs, Inputs).

9. System Requirements

9.1 Hardware Requirements

Component	Specification	Purpose
Processor	Intel Core i3 or higher	For smooth performance during design and execution.
RAM	Minimum 4 GB	Ensures efficient multitasking and tool performance.
Storage	500 GB or higher	To store project files, designs, and prototype data.

Component	Specification	Purpose
Internet	Stable Wi-Fi or LAN	Required for cloud access (Firebase/Cloudinary) and collaboration.
Mobile	Smartphone / Tablet (Optional)	To test mobile accessibility and user experience.

9.2 Software Tools & Technologies

Software Tool	Purpose / Description
Next.js	Framework allowing a mix of server-rendered and client-side components for better performance.
TypeScript	Adds type safety to JavaScript code, preventing common bugs.
Firebase	**Auth:** User sign-in. **Firestore:** Real-time database. **Storage:** Secure raw file storage.
Cloudinary	Handles direct client-side image uploads and delivery optimization.
Genkit	Uses Google's Gemini model to power the "theme suggestion" feature.
ShadCN UI	Provides well-designed, accessible components (Cards, Buttons, Dialogs).

10. Feasible Solution & Implementation Plan

10.1 Technical Approach

The solution leverages a **Serverless** architecture to minimize maintenance costs and maximize scalability.

- **Frontend:** Built with **Next.js** using the App Router. Pages are composed of **ShadCN** components tailored with Tailwind CSS.
- **Data Flow:**
    1. User logs in via **Firebase Auth**.
    2. User selects a photo -> Uploads directly to **Cloudinary** (for speed/optimization).

3. Cloudinary returns a URL -> App saves this URL + Metadata (Caption/Voice Note) to **Firestore**.
4. **Genkit** triggers on special dates to read Firestore metadata and suggest themes.

10.2 Implementation Steps

1. **Setup:** Initialize Next.js project with TypeScript and ShadCN UI.
2. **Auth:** Configure Firebase project and integrate Authentication hooks.
3. **Database:** Design Firestore schema (Collections: families, users, photos).
4. **Media:** Set up Cloudinary upload presets and integrate the upload widget.
5. **AI Integration:** Configure Genkit with Gemini API key for theme logic.
6. **Testing:** Verify security rules in Firestore and mobile responsiveness.

11. Future Enhancements

- **Video Clips:** Expand Cloudinary usage to support video streaming.